

Complejidad Temporal

Tarea 5

Alumno: Carlos Saúl Paz Maldonado

Para cada uno de los siguientes métodos, determine la función de tiempo $T(n)$ y la función de espacio $S(n)$.

Toma en cuenta que n representa la cantidad de datos del arreglo.

1.-

```
public static int sumaPares(int[] datos) {  
    int suma = 0;  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] % 2 == 0) {  
            suma += datos[i];  
        }  
    }  
    return suma;  
}
```

$S(n) = 1$ (for `int suma = 0;`) $T(n) = 1$

$S(n) = 1$ (for `for` loop) $T(n) = n$

$S(n) = 0$ (for `if` statement) $T(n) = \log_2 n$

$S(n) = 0$ (for `suma += datos[i];`) $T(n) = 1$

$S(n) = 0$ (for `return suma;`) $T(n) = 1$

$T(n) = 1 + n + \log_2 n + 1 + 1 = 3 + n + \log_2 n$

Suma todos los elementos pares de un arreglo.

2.-

```
public static int contarApariciones(int[] datos, int buscado) {  
    int contador = 0;  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] == buscado) {  
            contador++;  
        }  
    }  
    return contador;  
}
```

$S(n) = n$ (for `int[] datos`) $T(n) = 1$

$S(n) = 1$ (for `int contador = 0;`) $T(n) = 1$

$S(n) = 1$ (for `for` loop) $T(n) = n$

$S(n) = 1$ (for `if` statement) $T(n) = 1$

$S(n) = 0$ (for `contador++;`) $T(n) = 1$

$S(n) = 0$ (for `return contador;`) $T(n) = 1$

$S(n) = n + 1 + 1 + 1 = 3 + n$ **$T(n) = 3 + n$**

Cuenta las veces que aparece un numero específico en un arreglo.

3.-

```

public static int[] obtenerCuadrados(int[] datos) {
    int[] cuadrados = new int[datos.length];

    for (int i = 0; i < datos.length; i++) {
        cuadrados[i] = datos[i] * datos[i];
    }

    return cuadrados;
}

```

Annotations for problem 3:

- $S(n) = n$ (points to `datos`)
- $S(n) = n$ (points to `cuadrados`)
- $T(n) = 1$ (points to `new int[datos.length]`)
- $S(n) = 1$ (points to `i`)
- $T(n) = n$ (points to the `for` loop)
- $S(n) = 0$ (points to `cuadrados[i]`)
- $T(n) = 1$ (points to `datos[i] * datos[i]`)
- $S(n) = 0$ (points to `return`)
- $T(n) = 1$ (points to the `return` statement)

$T(n) = 3+n$

$$S(n) = n+n+1 = 1+2n$$

Eleva al cuadrado todos los elementos de un arreglo y los devuelve en otro array.v

4.-

```

public static boolean misteriosa(int[] datos, int x) {
    for (int i = 0; i < datos.length; i++) {
        for (int j = i + 1; j < datos.length; j++) {
            if (datos[i] + datos[j] == x) {
                return true;
            }
        }
    }

    return false;
}

```

Annotations for problem 4:

- $S(n) = n$ (points to `datos`)
- $S(n) = 1$ (points to `x`)
- $S(n) = 1$ (points to `i`)
- $T(n) = n$ (points to the first `for` loop)
- $S(n) = 0$ (points to `j`)
- $T(n) = n$ (points to the second `for` loop)
- $S(n) = 0$ (points to `datos[i]`)
- $T(n) = 1$ (points to `datos[j]`)
- $S(n) = 0$ (points to `return true`)
- $S(n) = 0$ (points to `return false`)
- $T(n) = 1$ (points to the `return false` statement)

$T(n) = n^2+1$

$$S(n) = n+1+1+0+0+0 = 2+n$$

Busca si existen dos numeros, en el array, que sumados entre sí den como resultado el número que se le paso por parámetro.

5.-

```

public static int misteriosa2(int[] datos) {
    int contador = 0;

    for (int i = 0; i < datos.length; i++) {
        for (int step = 1; step < datos.length; step *= 2) {
            contador += datos[i];
        }
    }

    return contador;
}

```

Annotations for problem 5:

- $S(n) = n$ (points to `datos`)
- $S(n) = 1$ (points to `contador`)
- $T(n) = 1$ (points to `contador = 0`)
- $S(n) = 1$ (points to `i`)
- $T(n) = n$ (points to the first `for` loop)
- $S(n) = 1$ (points to `step`)
- $T(n) = \log_2 n$ (points to the second `for` loop)
- $S(n) = 0$ (points to `contador`)
- $T(n) = 1$ (points to `datos[i]`)
- $S(n) = 0$ (points to `return`)
- $T(n) = 1$ (points to the `return` statement)

$$S(n) = n+1+1+1+0+0 = 3+n$$

$$T(n) = 1+n\log_2 n+1+1 = 3+n\log_2 n$$

Solo suma los elementos internos y los multiplica por la cantidad de datos en el array menos 1 (A la cantidad de datos se le resta 1 y ese se multiplica por la suma de los elementos del array).